# 8-Week Software Engineering Internship Plan

A hands-on track focused on shipping two internal tools end to end — not just tutorials.

## What this internship is for

Over eight weeks, the intern will design, build, test, and document two internal products. The aim is to leave with working software and a clear sense of how production systems are put together — architecture, APIs, data, auth, integrations, and deployment — not a pile of unfinished demos.

**1. Engineering Performance & Reporting Platform**

Automates how we pull GitHub activity into weekly performance reports, with AI-assisted summaries and an approval path for managers.

**2. No-Code AI / ML Learning Platform**

A guided environment where team members can try classification, regression, time-series, and LLM workflows without writing code first.

## Recommended stack

Keep the stack consistent across both products so time goes into product work, not tooling churn.

**BACKEND**

Python, FastAPI, PostgreSQL, SQLAlchemy, Alembic, Redis, Celery (or equivalent background jobs), JWT auth

**FRONTEND**

Vue 3 + Nuxt, TypeScript, Tailwind CSS, TanStack Query, Zod for form/API validation

**INTEGRATIONS**

GitHub API (REST + webhooks), OpenAI API for report generation and learning assistants

**DEVOPS**

Docker, Docker Compose, GitHub Actions for CI

*Frontend stays on Vue/Nuxt for both products. Shared UI patterns and components where it makes sense.*

## Week-by-week plan

**WEEK 1**

### Foundations & authentication service

Settle into the repo workflow (GitHub Flow, small PRs, review comments) and stand up the shared backend skeleton: FastAPI, PostgreSQL, Docker Compose, and a basic config story with environment variables.

Main build for the week: registration, login, JWT access + refresh tokens, and simple role checks. Swagger should stay current as endpoints land.

> **Done when:** Auth endpoints work end to end in Docker, the DB is wired up, and the intern can walk through the login/refresh flow in a short demo.

WEEK 2

### Reporting platform backend

Expand the API around the reporting domain: users, teams, organisations, reports, approvals, and comments. Focus on clean module boundaries (routes → services → data access), validation, sensible errors, and pagination/filtering where lists will get large.

Spend real time on the schema — relationships between users, teams, reports, and approvals should be deliberate, not bolted on later. Alembic migrations from day one.

> **Done when:** CRUD for the core modules is in place, an ERD has been reviewed with the team, and unit tests cover the critical service paths.

---

WEEK 3

### GitHub sync & scheduled jobs

Connect engineer accounts to GitHub (OAuth), then pull repositories, commits, pull requests, reviews, and issues into our database. Handle rate limits properly; don't pretend the happy path is enough.

Add webhooks where useful and a daily sync job (Celery/cron) so data doesn't depend on someone clicking "refresh." A simple engineer activity view on top of the synced data closes the loop.

> **Done when:** Live GitHub data syncs on a schedule and shows up correctly in the dashboard for at least one connected account.

---

WEEK 4

### AI-assisted reports

Use the synced activity to draft weekly summaries — manager-facing, exec-lite, and next-week goals. Prompts should produce structured output we can edit, not a black box dump into the database.

Wire the draft → edit → approve path, plus PDF export and email notification when a report is ready for review. Keep an eye on tokens and failure modes (empty weeks, sparse activity, API timeouts).

> **Done when:** A report can be generated from real GitHub data, edited, approved, exported to PDF, and the reviewer gets notified.

---

WEEK 5

### No-code AI platform — shell & onboarding

Start the second product. Ship a usable Nuxt frontend: dashboard, learning paths, CSV upload with preview, and a few sample datasets so people aren't stuck without data on day one.

Sketch the canvas / module navigation early, even if workflows are still light. Onboarding should get a new user to their first dataset without a walkthrough from engineering.

> **Done when:** The UI is navigable, uploads work, and a first-time user can get through onboarding without hand-holding.

WEEK 6

### Visual ML workflows

Implement guided workflows for the common cases we care about: tabular prediction from CSV, basic image classification, a simple time-series forecast, and an LLM playground (prompt testing, captioning, Q&A).

Preprocessing and feature choices should be visible in the UI — the point is learning, not hiding every step behind "Run." If time allows, a side-by-side comparison of two workflow runs is a nice extra, not a requirement.

> **Done when:** At least one full path (data in → train/run → result) works for tabular and one other modality.

WEEK 7

### From canvas to code

Bridge the gap between the no-code view and real code. From a completed workflow, generate readable Python the learner can inspect — classification, regression, time series, image, or LLM path depending on what they built.

Useful extras: side-by-side canvas vs code, export to a notebook or script, basic charts/metrics, and a short experiment history so runs aren't lost when the tab closes.

> **Done when:** Several workflow types can produce exportable code that actually runs, and the mapping from canvas steps to code is clear in a demo.

WEEK 8

### Hardening, docs, and handoff

Treat both products like something the team might keep running: Compose setups, CI on GitHub Actions, logging, basic security checks, and tests tightened around the risky paths (auth, sync, report generation).

Finish with docs people will actually use — setup guide, short architecture notes, API overview, and a user-facing walkthrough for each product. Close with a presentation covering design choices, what broke, what you'd do next, and a live demo of both tools.

> **Done when:** Both apps run from documented Compose setups, CI is green on main paths, and the final demo stands on its own without a "you had to be there" setup.

## Deliverables at a glance

| Week | Primary deliverable |
|---|---|
| 1 | Dockerised auth API with PostgreSQL and JWT (access + refresh) |
| 2 | Reporting domain APIs + reviewed database design |
| 3 | GitHub OAuth, sync jobs, and engineer activity view |
| 4 | AI report drafts, approval flow, PDF export, notifications |
| 5 | Learning platform UI, uploads, and first-run onboarding |
| 6 | Visual workflows for tabular + at least one other task type |
| 7 | Canvas → Python/notebook export with run history |

| Week | Primary deliverable |
|---|---|
| 8 | Deployment docs, CI, and final presentation |

## Weekly rhythm

- Short daily check-in (about 15 minutes)
- One pairing session each week
- PR reviews before anything merges
- Architecture check-in mid-week when design is shifting
- Aim for solid test coverage on new backend work (target ~80% on touched modules)
- Friday demo to engineering leads
- Brief notes on what was learned / blocked
- Quick retro + plan for the following week

## What “good” looks like by week 8

1. Can design and ship FastAPI services with a clear service layer and migrations.
2. Comfortable building Vue/Nuxt UIs in TypeScript that talk to those APIs reliably.
3. Can model relational data for multi-team reporting without painting us into a corner.
4. Has shipped a real GitHub integration (OAuth, sync, rate-limit awareness).
5. Can build LLM features that are editable and reviewable — not one-shot magic output.
6. Has a working starter for no-code ML learning (tabular + at least one other modality).
7. Can explain how a visual workflow maps to generated Python a beginner could read.
8. Leaves both apps runnable via Docker, with CI and docs another engineer can follow.